

CASPER@night — Installation and First-Run Guide

(Bruker / TopSpin)

John W. Blanchard (jwbquantum@gmail.com) Reza Ebadi Claude (Anthropic)

Software v0.7.9 — October 2026

1 What this is

CASPER@night is a community measurement program: any NMR facility with a Bruker spectrometer can contribute a data point with one 5 mm tube of water and one TopSpin command, `xpy spin_noise_run`. The script records the spin-noise feature of water — the Guéron absorption dip on room-temperature probes, the emission bump on cryoprobes — together with the receiver calibrations needed to interpret it absolutely, and packs everything into a single zip for upload. Across many facilities these bundles map the temperature-contrast law of circuit-spin coupling over the community's probe fleet, and they bank spin-noise-limited noise records for fundamental-sensitivity and dark-matter (CASPER-style) analyses.

The project page is night.blanchard-science.com. The data policy (`DATA_POLICY.md` in the distribution) is short and worth the two minutes — your raw data remain yours, and contributing facilities are co-authors on any network publication their data enter.

The five-minute version

1. Download the latest release zip: <https://github.com/CASPER-Night/spin-noise-network/releases/latest>
2. Copy three files onto the spectrometer workstation: `topspin/spin_noise_run.py` → `<TSHOME>/exp/stan/nmr/py/user/`, and `topspin/pp/zgnoise2d` together with `topspin/pp/zgref2d` → `<TSHOME>/exp/stan/nmr/lists/pp/user/`
3. Fill a 5 mm tube with $\sim 550 \mu\text{L}$ of plain water (any water — you just have to know what it is), insert it, and open any existing ^1H dataset in TopSpin.
4. Dry run first, no hardware touched: `xpy spin_noise_run simulate`
5. Real run: `xpy spin_noise_run` — answer the dialogs, **confirm the BSMS field sweep is OFF**, walk away (~ 45 min default).
6. Upload from any machine with Python 3: `python3 uploader/upload_bundle.py <bundle.zip>` (first time: copy `config.example.json` to `config.json` and paste the endpoint and token the coordinator sent you).

2 Requirements

- **A Bruker spectrometer running TopSpin 2.x, 3.x, or 4.x.** (TopSpin 5.0, released 2026, is untested: Bruker documents the Jython layer this software runs in as a standard component alongside the newer CPython interface, so it is expected to work — but we have not verified it.)

If your console runs TopSpin 5, please tell us what happens: you would be our first.) The script is written for TopSpin’s embedded Jython interpreter and avoids anything version-specific — every optional command (`atma`, `topshim`, `pulseval`, `rga`) degrades to an operator dialog when missing, so old consoles work too.

- **Any ^1H -capable probe** — room-temperature, nitrogen-cooled, or helium cryoprobe. The run asks which and records it.
- **One 5 mm tube of plain water**, $\sim 550\ \mu\text{L}$. Tap, distilled, or D_2O -doped all work — *as long as you know the composition*. The H_2O -fraction question in the dialogs is the measurement itself (the spin-noise feature scales with proton density), so if the tube contains D_2O , know the percentage.
- **Python 3.6 or newer on any machine that can reach the internet**, for the upload step only. The spectrometer host itself needs no internet access and no Python packages — carry the zip on a USB stick if needed. Check with:

```
python3 --version
```

- **NTP time synchronization enabled on the TopSpin workstation** (it usually is by default). The run measures your console’s master-clock offset for free by comparing the NTP-disciplined workstation clock against the console-timed acquisition durations — the “clock audit” in the bundle. Nothing on the spectrometer is touched or reconfigured — if your workstation is air-gapped, just mention that when you upload.
- **Disk and time:** the default run writes roughly 200 MB (≥ 10 GB free is comfortable headroom for reruns and longer noise blocks) and takes ~ 45 minutes of magnet time, of which you are needed for the first ~ 6 (the dialogs, then the one-minute row probe that follows the gain ladder).

3 Download

Get the latest release zip from the GitHub releases page:

```
https://github.com/CASPER-Night/spin-noise-network/releases/latest
```

The zip is the complete `spin-noise-network` distribution: the TopSpin acquisition kit, the uploader, the metadata schema, the protocol and policy documents, and testing tools. Only three files go onto the spectrometer workstation (next section):

- `topspin/spin_noise_run.py` — the acquisition script (Jython, runs inside TopSpin).
- `topspin/pp/zgnoise2d` — the no-pulse pseudo-2D pulse program used for the noise blocks (and the row probe). Since v0.7.7 it defines `acqt0=0` (Bruker’s idiom for a pulse-free acquisition), both pulse programs write each row during a data-transfer delay `d11` (1 s), and the script acquires every experiment in DIGMOD `digital` / DSPFIRM `sharp`: Oulu’s Avance III HD, running `baseopt` from the operator’s parameter set, had every 1 MB pseudo-2D row aborted by its receiver unit after the first (2026-09-30; confirmed on 2026-10-02, when the refused row acquired at the first try in `digital`). Since v0.7.8 the mode is read before it is written: one DIGMOD write per session and no DSPFIRM write on a console that couples the two, as TopSpin 3.7.0 does.
- `topspin/pp/zgref2d` — the small-flip pseudo-2D pulse program used for the two reference blocks (since v0.7.6): Bruker’s `zg2d` without the pacing delay it computes from `d20`, which the script never sets — on Torino’s Avance Neo that delay came out negative and the console refused to compile `zg2d` (2026-09-25), so the references now have a sequence of their own.

Keep the rest of the unpacked folder around on whatever machine will do the upload: the uploader,

uploader/upload_bundle.py, expects `schema/meta.schema.json` one level up from itself, as in the release layout. If you move the uploader out on its own, pass `--schema <path>` explicitly — otherwise its `--selftest` skips schema validation with a WARN yet still prints `RESULT: PASS`.

4 Install (three file copies)

1. Copy `spin_noise_run.py` to `<TSHOME>/exp/stan/nmr/py/user/`. Alternative: in TopSpin type `edpy`, use *File* → *Import...* and pick the file — that lands it in the same place.
2. Copy `pp/zgnoise2d` and `pp/zgref2d` to `<TSHOME>/exp/stan/nmr/lists/pp/user/`.
3. There is no step 3. No Python packages, no licenses, no network access is needed on the spectrometer.

`<TSHOME>` is your TopSpin installation directory — `/opt/topspin4.1.4` or `/opt/topspin3.6.5` on Linux, `C:\Bruker\TopSpin4.1.4` on Windows, etc. If you are unsure, type `set` inside TopSpin or look at the title bar of the TopSpin window.

The script also installs both pulse programs automatically on first run if it can find your TopSpin directory, so step 2 is a belt-and-braces copy — do it anyway if you can.

Verify TopSpin sees the script: type `edpy` and `spin_noise_run` should appear in the *user* source directory listing. The pulse programs are checked again at run time: if `zg` cannot find one, the script tells you and detects the file once you copy it in by hand (Section 8).

5 Verify without touching hardware

Two test modes let you exercise the whole flow before the magnet is involved. Both are started from the TopSpin command line with a **1D** ^1H dataset open (a PROTON demo set is fine — it is only a parameter template; since v0.7.4 the script sets the dimensionality of every dataset it creates itself, but a 1D template is the clean start). Either template works from v0.7.5: neither TopSpin 3.7.0 nor 4.4.0 creates the F1 parameter file when a script switches a dataset to 2D (Oulu and Torino, September 2026), so the script brings `acqu2` from your template dataset if that is 2D, else from another experiment of the session, else from the console's own parameter library, and `meta.json` records which (`software.param_api.f1_files_source`). In the F1 file it acquires with — copied or inherited — it sets `FnMODE` to *undefined*, Bruker's rule for a pulse program without an `mc` statement (`f1_fnmode_edits` counts the files it edited).

5.1 SIMULATE — the pure dry run

```
xpy spin_noise_run simulate
```

SIMULATE skips all spectrometer commands (and the setup blocks that lead to them) while still running every dialog, the dataset bookkeeping, the `meta.json` writing, and the zip bundling. Answer the dialogs with test values (suggested: institution `Desk Test Lab`, slug `desktest-lab`, sample `distilled water`, H_2O 100, D_2O 0, duration 30 min, lock OFF, sweep confirmed OFF). Success looks like:

- the greeting dialog shows `*** SIMULATE MODE ... ***`.
- all operator dialogs appear, in the order listed in Section 6.
- the terminal shows `SIMULATE -> ... / SIMULATE: zg mocked ...` lines and no errors.
- the final dialog reports the bundle path, and the zip exists there.

5.2 DESKTEST — the stronger check

```
xpy spin_noise_run desktest
```

DESKTEST drives the *real* TopSpin API — the full dialog chain, GETPAR/PUTPAR on live parameter sets, WR/RE dataset creation, `meta.json` writing, zip bundling — and mocks only the five hardware commands (`atma`, `topshim`, `pulsecal`, `zg`, `rga`). It runs fine on a processing-only TopSpin install (Bruker’s free academic license), so you can do it at your desk. Pass criteria, from the release’s desk-test checklist:

- terminal shows DESKTEST -> mocked `'atma'`, `'topshim'`, `'pulsecal'`, `'rga'` — and no manual-fallback dialog appears for any of them, and no Jython traceback anywhere.
- a dataset SPINNOISE_<yyyymmdd>_<hhmm> exists with expnos **1, 10, 14, 15, 16, 17, 11, 12, 13**, each holding the raw-data file its mocked acquisition left (since v0.7.7: `fid` for the 1Ds, `ser` for the pseudo-2Ds, of the blocks’ real size — about 100 MB for a 30-min desktest — filled with pseudo-random samples, never data; the status line says `row probe: attempt 1 passed`).
- `meta.json` is written twice (dataset directory and bundle) and contains a `software` block with `"script_version": "0.7.9"` (this release), `"schema_version": "1.2"` (the orchestrator’s Bruker-only metadata vintage, accepted under the repository’s vendor-neutral v2.0 schema), a `script_sha256` fingerprint, and `"run_mode": "desktest"` — plus a `clock_audit` block with one timing entry per acquisition (the row probe included) and a `calibration.row_probe` record.
- the bundle zip, SPINNOISE_<date>_<time>/spinnoise_<slug>_<YYYYMMDD_HHMMSS>Z_<hex>.zip, opens in any zip tool, and the validator passes on any Python-3 machine:

```
python3 uploader/upload_bundle.py <bundle.zip> --selftest  
-> RESULT: PASS
```

Never upload a SIMULATE or DESKTEST bundle — it contains copies of your demo dataset, not noise data. `meta.json` flags it, but do not rely on the flag — just do not send it.

6 The real run

One honest note before the first live run: as of v0.7.8 the script has been desk-tested end to end (a real Jython interpreter against stubbed TopSpin APIs modelling thirteen console behaviours, plus the SIMULATE/DESKTEST modes above), has completed DESKTEST on a real TopSpin 4.4.0 console (Torino, September 2026), where it found two console-specific faults that were fixed within a day, and on a TopSpin 3.7.0 console (Oulu, September 2026), where it found a third (the console did not create the F1 parameter file and silently dropped every parameter write into the 2D datasets; fixed in v0.7.5), and has run one live session (Torino, 2026-09-25): the pulse-free noise block acquired and the report resolves the water spin-noise dip on it, but both reference blocks were refused by the console because Bruker’s `zg2d` computes a pacing delay from a parameter the script never set — fixed in v0.7.6 by the project’s own `zgrep2d`. Oulu’s first live run with v0.7.6 (2026-09-30, Avance III HD 500, TopSpin 3.7.0) then met the receiver unit: every pseudo-2D block acquired its first 1 MB row and was aborted 12–18s into the second (“too much data for the LAN capacity”) while the small 1D rungs passed, and the console left full-size `ser` files holding one acquired row and zeros, which the script took for data. Every Oulu expno ran DIGMOD `baseopt` from the operator’s parameter set — a mode in which Bruker documents 16 times the points processed inside the DRU — while Torino’s Neo had acquired the same rows in `digital`: the leading explanation at the time, fixed in v0.7.7 by

setting **digital** / **sharp** on every experiment, with the data-transfer delay **d11** as insurance, the row probe as the safety net and the content check of every block. On 2026-10-02 the operator’s manual check at Oulu settled it: the row the console had refused in **baseopt** acquired at the first try in **digital**. The same day the v0.7.7 desktest there completed cleanly but popped the console’s own **GetEnuOrd[DSPFIRM]** dialog once per experiment — harmless; v0.7.8 reads the mode before writing it and no longer writes **DSPFIRM** on such a console. A complete live session, references included, is still pending. We therefore treat the first live run at each facility as a supervised pilot — write to jwbquantum@gmail.com and we will screen-share through it.

6.1 Before you type the command

Insert the water tube (sample changer or manual — the script never ejects or injects), set your usual VT setpoint, and let the sample equilibrate for a few minutes. Then, in this order: **lock**, **shim**, **tune** as you usually do (lock on), then **lock OFF** and **BSMS field sweep OFF**, and only then start the script. The script still calls **atma** and **topshim** where your system has them, but **topshim** refuses to run while the lock is off (*lock is off: please lock in prior to shimming* — Torino, 2026-09-22) and this protocol keeps the lock off, so its shimming dialog will appear: you shimmed already — press OK. Where a command is missing it walks you through the manual equivalent (**wobb**, your usual shim routine). Open any existing ^1H dataset, then:

```
xpy spin_noise_run
```

6.2 The dialogs, in order

1. **Welcome** — summarizes what the run will do (Start/Cancel).
2. **Your facility (1/5)** — institution, city, country, contact e-mail, so the bundle is attributed correctly.
3. **Facility slug (2/5)** — short machine-readable ID (auto-suggested), used in the bundle file name.
One slug per instrument: a site with two magnets is two network nodes (e.g. **uni-oulu** and **uni-oulu400**), each run under its own slug; one **config.json** serves both at upload time (the uploader only warns when a bundle’s slug differs from the configured one).
4. **Contact consent** — whether the maintainers may contact you at that address. The e-mail is stored only if you answer yes.
5. **The sample (3/5)** — description, H_2O fraction (%), D_2O fraction (%), additives, tube OD, volume. Answer exactly as the tube is. Tap water with a described history beats an undocumented perfect sample.
6. **Temperature (4/5)** — VT setpoint in K (enter 298 if VT is off and the bore is at room temperature).
7. **Noise-block duration (5/5)** — 30 min (default), 60 min, 180 min, or overnight (10 h). Setup and references add ~ 15 min on top. Longer runs average down the spectrum, and an overnight run on an idle weekend magnet is gold.
8. **Lock state** — turn the lock OFF for the noise block if you can (**lock off** / BSMS LOCK key): lock rf leaks into the ^1H channel on some systems. Either answer is accepted and recorded.
9. **BSMS field sweep — the one that matters.** Open **bsmsdisp** and *physically verify* SWEEP is OFF before answering. A sweeping field smears the spin-noise feature over several kHz and leaves a perfectly healthy-looking flat noise floor — this silently contaminated an archival dataset in 2022, and the script cannot verify the sweep state on all console generations, so the

confirmation is on you. If you cannot confirm, a follow-up dialog offers proceed-flagged or abort. Prefer fixing it to proceeding flagged.

10. **Hardware check** — TopSpin version, console, probe string (pre-filled where the script can detect them) — correct as needed.
11. **Probe type** — room-temperature, nitrogen-cooled, helium cryoprobe, or unknown (this sets which side of the temperature-contrast law your point lands on).
12. **Probe temperatures (optional)** — coil and preamp temperatures in K if you know them (`edte` / `cryopanel`). Blank is fine.
13. **90-degree pulse** — confirm P90 (μs) and its power (dB), pre-filled from `pulsecal` or the current P1. Sanity-check it ($\sim 7\text{--}15\ \mu\text{s}$ is typical) before confirming.
14. *Row probe* (v0.7.7, status line only, ~ 1 min) — after the four gain-ladder rungs the script acquires a short pulse-free pseudo-2D at expno 17, in the `digital` acquisition mode it has set, and checks that its *last* row holds data; the status line says `row probe: attempt 1 passed`. A console whose receiver unit refuses the default row even so makes it try a longer transfer delay and then shorter rows (up to ~ 5 min); the setting it settles on is recorded in `meta.json` (`calibration.row_probe`) and used for every block — a console property, not a fault. Stay until you have seen the `passed` line.
15. *Noise block auto-start* — after the references the script prints the row count, per-row time, and RG to the status line and starts the pulse-free block by itself 30 s later. There is no dialog to click: *the row probe's "passed" line is the walk-away point* (the opening reference runs unattended after it, ~ 3 min, then the noise block).
16. **Final notes** — anything unusual during the run (construction, elevator, He fill, lock left on, ...). Blank if all was quiet.
17. **Done** — the bundle path and the one-line upload command.

Two conditional extras: if the script cannot find your TopSpin directory it asks once for the user pulse-program path, and on systems missing `atma/topshim/pulsecal/rga` it asks you to perform that one step manually and press OK.

6.3 What it does, and what it will not touch

The run creates one dataset, `SPINNOISE_<date>_<time>`, in your current data directory:

expno	role
1	setup: tune/match, shim, P90 calibration
10, 14, 15, 16	RG ladder: quick 1° 1Ds at RG = 1, 8, 64, max
17	row probe: <code>zgnoise2d</code> , RG 1, two or more rows per attempt — the row length / transfer delay th
11	reference (open): <code>zgref2d</code> , 1° pseudo-2D, ~ 170 s of rows ($8 \times \sim 21$ s at the default row)
12	noise : <code>zgnoise2d</code> , <i>no pulse at all</i> , NS=1/row, RG max stable
13	reference (close): same as 11 (drift check)

Pulsing is limited to the standard pulse calibration and $\sim 1^\circ$ tips at the calibrated observe power — no long or repeated high-power irradiation. The script never changes instrument configuration, never touches your lock or sweep settings silently (it asks you and records your answer), and reads and writes nothing outside the `SPINNOISE_<date>_<time>` dataset plus the three installed files. Complete removal, if you ever want everything gone:

```
rm -rf <DATADIR>/SPINNOISE_<yyyymmdd>_<hhmm>
rm <TSHOME>/exp/stan/nmr/py/user/spin_noise_run.py
rm <TSHOME>/exp/stan/nmr/lists/pp/user/zgnoise2d
rm <TSHOME>/exp/stan/nmr/lists/pp/user/zgref2d
```

(Windows: delete the same four paths in Explorer. <DATADIR> is the data directory of the template dataset you had open.)

The final dialog prints the full path of the finished bundle, which lands inside the SPINNOISE_<date>_<time> dataset directory as spinnoise_<slug>_<timestamp>Z_<hex>.zip. You keep this local copy no matter what happens during upload.

7 Upload

7.1 One-time setup

On whichever machine will do the upload (Python $\geq$ 3.6, internet), go to the `uploader/` folder, copy `config.example.json` to `config.json`, and fill in the values **the coordinator sent your facility privately**:

```
{
  "endpoint_url": "https://...workers.dev/ingest",
  "token":        "<from the coordinator>",
  "facility_slug": "your-facility-slug"
}
```

The endpoint and token arrive by a private channel, never in e-mail chains — treat the token like a password. `config.json` is read locally and never uploaded.

7.2 The upload

```
python3 uploader/upload_bundle.py /path/to/spinnoise_*.zip
```

Size never matters: bundles up to 50 MiB go in one request, and anything larger (overnight runs, up to 5 GiB) automatically switches to a chunked upload in 50 MiB parts. Progress is checkpointed next to the zip (<bundle>.upload-state.json), so if the network drops or the machine reboots, **rerun the same command and it resumes where it left off**. The uploader never deletes your zip.

Success ends with a line like:

```
RECEIPT: <id>
```

That is the server confirming your bundle arrived intact (its SHA-256 was verified on arrival) and was recorded. Note the id — it is the reference for any follow-up about this bundle.

7.3 First test run, or no endpoint yet

If you have no `config.json` (or no internet path at all), the script prints exactly what to do: the zip stays where it is, and you can e-mail or file-transfer it to jwbquantum@gmail.com (a share link is fine if it exceeds mail limits). For a first test you can also validate locally without any network:

```
python3 uploader/upload_bundle.py <bundle.zip> --selftest
```

8 Troubleshooting

- **“No dataset is open”** — open any ^1H dataset first (it is only the parameter template).
- **Pulse program not found at zg** — copy `pp/zgnoise2d` and `pp/zgref2d` into the user pulse-program directory (`<TSHOME>/exp/stan/nmr/lists/pp/user/`) by hand and rerun — the script will detect them.
- **TopSpin shows TCube: Cannot interpret pulse program: Cannot load line: duration is negative (-21166512.000000 us) In 'zg2d': line 24 as the opening reference (expno 11) starts, then the script asks about a missing raw-data file (script v0.7.5 or earlier)** — Bruker’s library `zg2d`, which the older script used for the two reference blocks, paces its rows with a delay computed as $d20 - (d1 + aq) - 30\text{ms}$; the script never set `d20`, so with $D1 = 2\text{s}$ and $AQ = 19.1\text{s}$ the delay was -21.17s and the console refused to compile the sequence. Torino’s first live run (2026-09-25) lost both reference blocks this way while the noise block acquired normally. Answer the script’s **acquisition check** with Cancel and install v0.7.6, whose `zgref2d` computes no delay. Workaround only, on an older script: type `d20 21.2` with the template dataset open before starting the run (each reference row then lasts 21.2s). See `docs/TROUBLESHOOTING.md`.
- **TopSpin shows Exception in DRUCONTR 1: Your pulse program produces too much data for the LAN capacity. ->Experiment aborted by DRU1! during a reference or noise block (script v0.7.6 or earlier; Avance III HD, TopSpin 3.7.0 at Oulu, 2026-09-30)** — the receiver unit acquired the first 1 MB row of each block completely and aborted the block 12–18s into the second; the console left full-size `ser` files of one row and zeros, which the old script took for data and packed. The cause, confirmed at Oulu on 2026-10-02: the acquisition mode — every Oulu expno ran `DIGMOD baseopt` (16 times the points inside the DRU, per Bruker), Torino’s Neo acquired the same rows in `digital`, and the refused row acquired at the first try once set to `digital` by hand; the 30–50ms write window before `wr` was the weaker one (the abort came mid-row). Install v0.7.8: the script sets `DIGMOD digital / DSPFIRM sharp` on every experiment, both pulse programs write during a 1s transfer delay `d11` (insurance), the row probe (expno 17) checks before the references that the row comes back with data and shortens it if not (safety net), and every block is checked for content afterwards — with only status-line WARNINGS, never a dialog, after the probe. That is what the manual check on the old dataset did (expno 12 of `SPINNOISE_20260930_1201`, `1 td → 1, digmod → digital, zg`). See `docs/TROUBLESHOOTING.md`.
- **TopSpin shows GetEnuOrd[DSPFIRM]: enumeration name sharp not found, once per experiment (script v0.7.7; TopSpin 3.7.0 at Oulu, 2026-10-02)** — harmless: the console had already moved `DSPFIRM` to `sharp` when the script set `DIGMOD digital`, and refused the script’s explicit `DSPFIRM sharp` (its enum table spells it `sharp(standard)`) with a dialog that raised no exception, so the script repeated it on every expno. Every dataset was acquired in `digital / sharp`; close the dialogs. v0.7.8 reads the mode before writing it and writes `DSPFIRM` only when the console did not couple. See `docs/TROUBLESHOOTING.md`.
- **TopSpin shows rga: acqt0 not set in pulse program, result may be incorrect! at the noise block (script v0.7.6 or earlier)** — harmless: the old `zgnoise2d` defined no time origin; `rga` still returned the gain that keeps the pulse-free rows clear of full scale. v0.7.7’s `zgnoise2d` defines `acqt0=0`, Bruker’s own idiom for a pulse-free acquisition, and the warning is gone.
- **A parmode dialog appears** — some TopSpin versions ask before converting a dataset to 2D. Answer yes/OK — the dataset is fresh, there is nothing to lose.

- **TopSpin shows an error about PARMODE / GetEnuOrd, then the script asks you to type parmode** — TopSpin validates parameters written from a script by name; the old script wrote a number. Script v0.7.3 or later writes the documented name, verifies it, and does the first switch while you are still at the console (at the start of the setup step), so at worst you type `parmode` in TopSpin (or use the `eda` toolbar button *Change data dimensionality*), select 2D, once. See `docs/TROUBLESHOOTING.md`.
- **TopSpin shows The requested format file is invalid: .../SPINNOISE_<date>/<expno>/acqu2: getpar: No such file or directory (script v0.7.4 or earlier; TopSpin 3.7.0 at Oulu and 4.4.0 at Torino, both with a 1D template)** — the console accepted the switch to 2D but did not create the F1 parameter file `acqu2`, and while it was missing every parameter write into that dataset was silently lost, with no error reaching the script (the pseudo-2D blocks kept the 1D template's TD/RG/pulse program); on 4.4.0 `zg` then refused each block with inconsistent PARMODE 2D ... `acqu2` — answer the script's **acquisition check** dialog with Cancel in that case. Do not run live with v0.7.4 on such a console; v0.7.5 creates the file itself and reads TD and RG back after writing. See `docs/TROUBLESHOOTING.md` for what to check in `meta.json`.
- **Old TopSpin (2.x)** — everything is written for Jython 2.2-level syntax. If `Avance.incl` is missing, delete the `#include` line in `zgnoise2d` and in `zgref2d` (neither uses it).
- **No ATM probe / no rga** — expected and handled: the script asks you to tune/match with `wobb` (or set RG) manually and confirm in a dialog.
- **A hardware command hangs with no progress for over 5 minutes** — kill the script window. Any acquisition already started finishes on its own and all data stays in `SPINNOISE_<date>_<time>`.
- **A Jython error dialog appears** — the run stopped, but started acquisitions finish and the data stays in `SPINNOISE_<date>_<time>`. Copy the error text verbatim into a support issue (below) — a stuck run reported well is worth almost as much as a clean one.
- **Facility Python 3 older than 3.6** — run the selftest and upload from any other machine (copy the zip on a stick). The uploader is stdlib-only by design, so please do not `pip-install` anything on the spectrometer workstation.

Where to get help: open a facility-support issue at <https://github.com/CASPER-Night/spin-noise-network/issues/new?template=facility-support.yml> (“don’t know” is always an acceptable answer on that form), or write to jwbquantum@gmail.com.

Data policy: your raw data remain yours, and contributing facilities are co-authors on network publications their data enter — full terms in `DATA_POLICY.md` and at night.blanchard-science.com.